

(Chapter 3 Producer in Kafka)

[topic,
value,
key,
partition,
<headers>] → Producer Record.

(Two types of Message Sending from Producer)

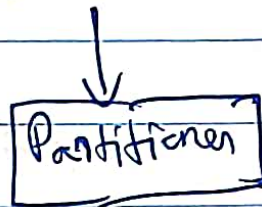
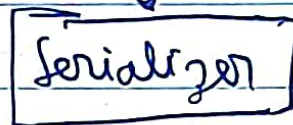
Synchronous send → • get() method returns
Future Object before sending
next record, to • get() call

Asynchronous send → Send it and a callback function
is triggered when response is
received

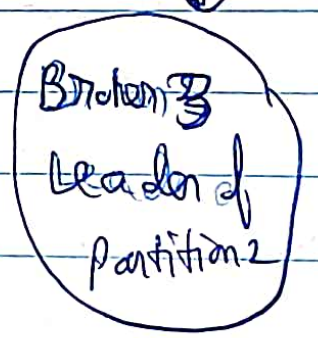
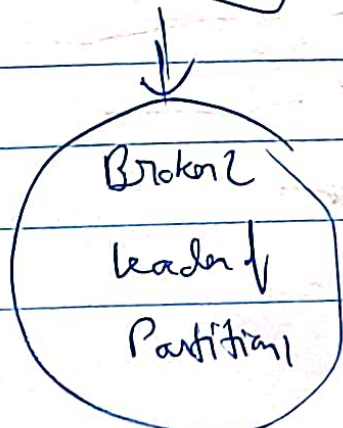
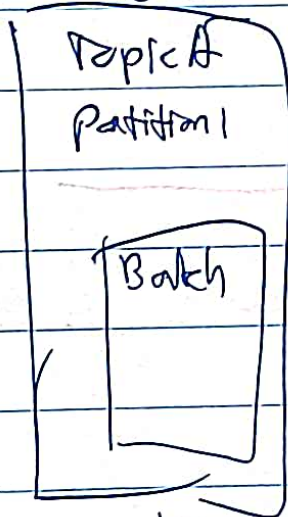
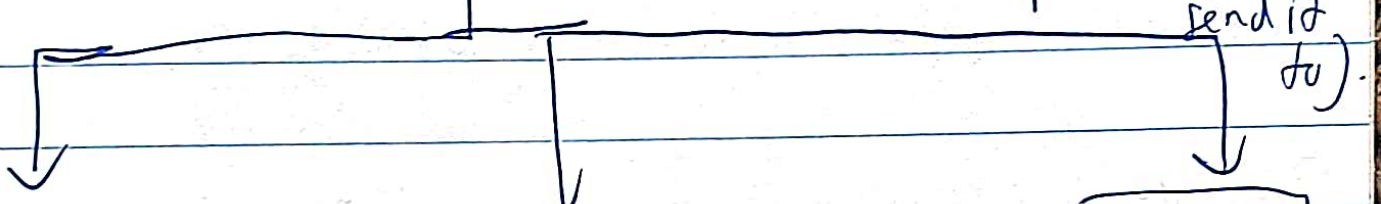
Broker record.



↓ send()



(decides on key which partition to send it to).



In asynchronous send callbacks ~~are~~ will be executed in the main thread, and hence callbacks will be executed in the order they are sent.

Producer conf/reqs

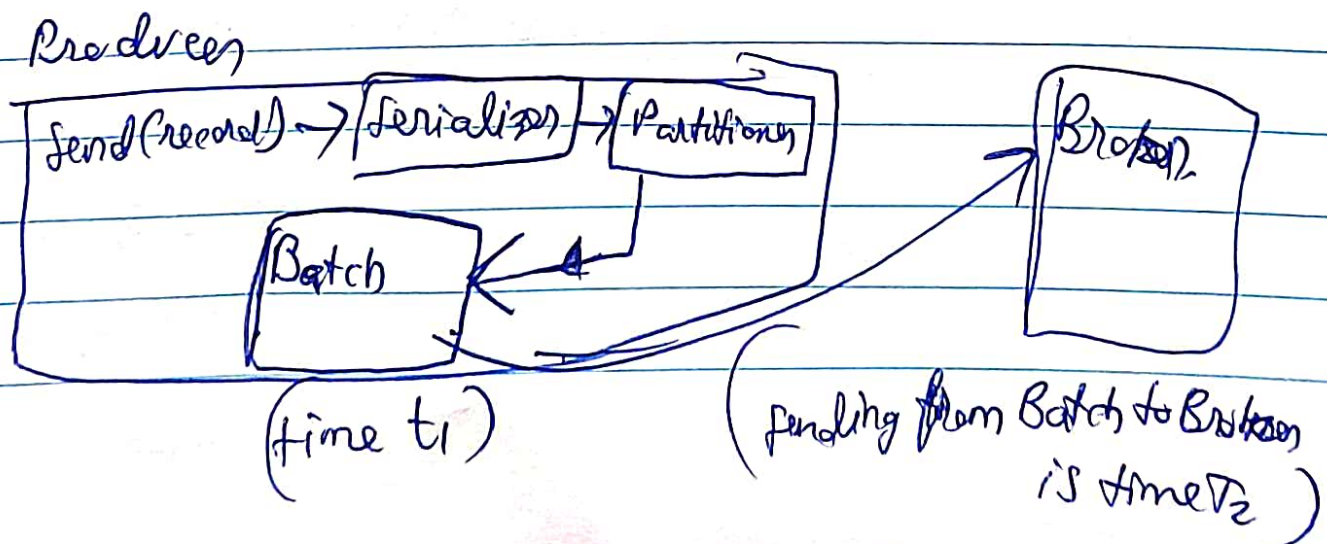
(acks = 0) fire and forget from producer

(acks = 1) leader's ack that write has happened

(acks = 99) (leader + 9 ISR) acks.

*** (Message Delivery Time)

2 steps message delivery is broken into.



max.block.ms $\rightarrow$ T1

request.time.out.ms $\rightarrow$ Max time allowed to
OS thread to send from
Batch to broker.

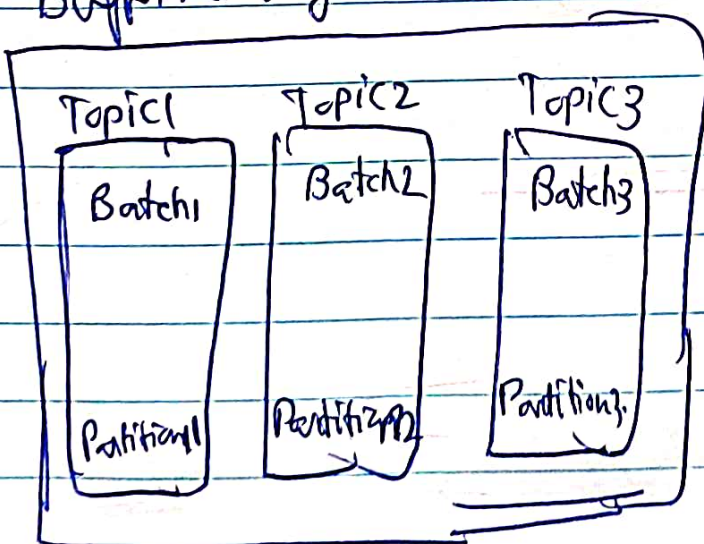
delivery.time.out $\rightarrow$ Now if OS thread fails to
send from batch to broker,
then retries are made.

And this

$$\left((\text{request.time.out.ms}) \times \text{retries} \right) = \text{delivery.time.out}$$

(Memory)

Buffer Memory



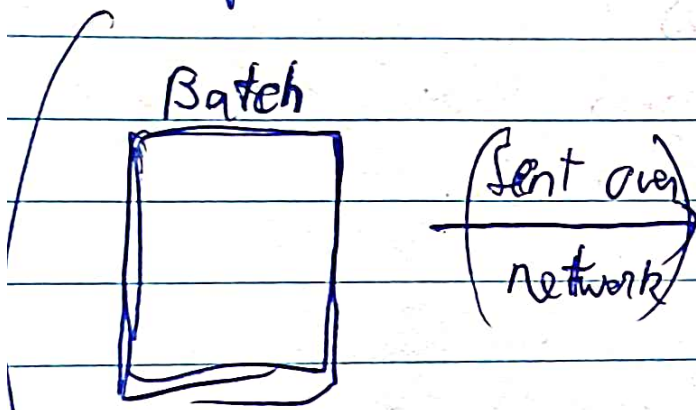
Buffer memory
entire
memory allocated
in RAM for
holding batches.

Batch:Size $\rightarrow$ max
Size after which
producer sends batch to
broker.

linger-ms $\rightarrow$ timeout after which batch should be sent from ~~producer~~ producer to broker.

~~(***)~~ V-imp.

max-inflight-requests-per-connection $\rightarrow$



If max-inflight-requests = 3, then 3 batches can be sent without waiting for acknowledgments.

Higher value Higher throughput
But ordering problems

(Kafka Order Guarantee)

With retries and

(high # of max-inflight-requests) ordering can be messed up.

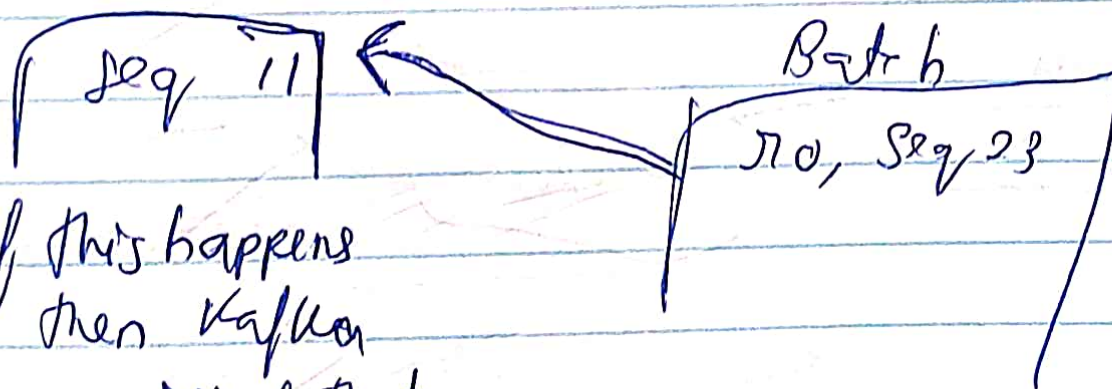
(enable.idempotence = true)

↳ avoids duplication and maintains ordering guarantee.

with every producer record a seq. No is assigned.

while processing a batch, the first record's seq no is checked if it matches then ~~that~~ that batch is processed,

let say broker is expecting--



if this happens then Kafka rejects that batch.

(Compression types)

(Snappy) → decent compression ratio
low CPU usage, when low
CPU overhead is required

GZIP → high compression ratio high CPU
overhead.

(AVRO Serialisation Use Case)

